

```
// src/services/drive.service.ts - VERSIÓN OAUTH2 CORREGIDA

import { google, drive_v3 } from 'googleapis';

import { OAuth2Client } from 'google-auth-library';

import { config } from '../config/env';

import logger from '../utils/logger';

import { Readable } from 'stream';

class DriveService {

  private drive: drive_v3.Drive | null = null;

  private oauth2Client: OAuth2Client | null = null;

  private initialized: boolean = false;

  constructor() {

    this.initializeDrive();

  }

  private initializeDrive(): void {

    try {

      // Verificar que tengamos las credenciales OAuth2

      if (!config.google.clientId || !config.google.clientSecret || !config.google.refreshToken) {

        logger.error('Faltan credenciales OAuth2 en el .env');

        this.initialized = false;

        return;

      }

      // Crear cliente OAuth2

      this.oauth2Client = new google.auth.OAuth2(

        config.google.clientId,

        config.google.clientSecret,

        'urn:ietf:wg:oauth:2.0:oob' // Redirect URI para aplicaciones de escritorio

      );

      // Configurar el refresh token

      this.oauth2Client.setCredentials({

        refresh_token: config.google.refreshToken

      });

      // Inicializar Drive API con OAuth2

      this.drive = google.drive({

        version: 'v3',

        auth: this.oauth2Client

      });

      this.initialized = true;

      logger.info('Google Drive service inicializado con OAuth2');

    }

  }

}
```

```
// Verificar que funciona obteniendo info de la cuota

this.verifyConnection();

} catch (error: any) {

  logger.error('Error al inicializar Google Drive:', {

    message: error.message

  });

  this.initialized = false;

}

private async verifyConnection(): Promise<void> {

  try {

    const response = await this.drive!.about.get({

      fields: 'storageQuota, user'

    });

    logger.info('Conexión verificada con Drive', {

      user: response.data.user?.emailAddress,

      quotaUsed: response.data.storageQuota?.usage,

      quotaLimit: response.data.storageQuota?.limit

    });

  } catch (error) {

    logger.error('Error verificando conexión:', error);

  }

}

async uploadFile(

  buffer: Buffer,

  filename: string,

  mimeType: string

): Promise<{ fileId: string; webViewLink: string }> {

  if (!this.initialized || !this.drive) {

    throw new Error('Google Drive service no está inicializado. Verifica las credenciales OAuth2.');
```

```
  }

  try {

    logger.info('Subiendo archivo a Google Drive con OAuth2', {

      filename,

      size: buffer.length,

      folderId: config.google.driveFolderId

    });

    // Metadata del archivo

    const fileMetadata: drive_v3.Schema$File = {

      name: filename,

      parents: [config.google.driveFolderId]
```

```

};

const media = {

  mimeType,

  body: Readable.from(buffer),

};

// Subir el archivo

const response = await this.drive.files.create({

  requestBody: fileMetadata,

  media,

  fields: 'id, webViewLink, webContentLink, name, size'

});

const fileId = response.data.id!;

const webViewLink = response.data.webViewLink ||
`https://drive.google.com/file/d/${fileId}/view`;

logger.info('Archivo subido exitosamente con OAuth2', {

  fileId,

  filename,

  webViewLink,

  size: response.data.size

});

return { fileId, webViewLink };

} catch (error: any) {

  logger.error('Error al subir archivo:', {

    message: error.message,

    code: error.code,

    errors: error.errors

  });

  // Intentar refrescar el token si es un error de autenticación

  if (error.code === 401) {

    logger.info('Token expirado, intentando refrescar...');

    try {

      const { credentials } = await this.oauth2Client!.refreshAccessToken();

      this.oauth2Client!.setCredentials(credentials);

      logger.info('Token refrescado exitosamente');

      // Reintentar la subida

      return this.uploadFile(buffer, filename, mimeType);

    } catch (refreshError) {

      logger.error('Error al refrescar token:', refreshError);

    }

  }

}

```

```

    throw new Error(`Error al subir archivo: ${error.message}`);

  }

}

async deleteFile(fileId: string): Promise<void> {

  if (!this.initialized || !this.drive) {

    logger.warn('Intento de eliminar archivo sin Drive inicializado');

    return;

  }

  try {

    await this.drive.files.delete({ fileId });

    logger.info('Archivo eliminado de Drive', { fileId });

  } catch (error: any) {

    logger.error('Error al eliminar archivo:', {

      message: error.message,

      fileId

    });

  }

}

async getFileMetadata(fileId: string): Promise<drive_v3.Schema$File | null> {

  if (!this.initialized || !this.drive) {

    return null;

  }

  try {

    const response = await this.drive.files.get({

      fileId,

      fields: 'id, name, mimeType, size, webViewLink, createdTime,
modifiedTime'

    });

    return response.data;

  } catch (error: any) {

    logger.error('Error al obtener metadata:', {

      message: error.message,

      fileId

    });

    return null;

  }

}

// Método auxiliar para listar archivos (útil para debug)

async listFiles(limit: number = 10): Promise<drive_v3.Schema$File[]> {

  if (!this.initialized || !this.drive) {

    return [];

  }

}

```

```

try {

  const response = await this.drive.files.list({

    pageSize: limit,

    fields: 'files(id, name, mimeType, size, parents, webViewLink)',

    q: ` '${config.google.driveFolderId}' in parents and trashed = false `

  });

  return response.data.files || [];

} catch (error: any) {

  logger.error('Error al listar archivos:', error.message);

  return [];

}

}

/**

 * Descarga un archivo de Google Drive como Buffer

 * @param fileId ID del archivo en Drive

 * @returns Buffer del archivo

 */

async downloadFileBuffer(fileId: string): Promise<Buffer> {

  try {

    // ===== VALIDACIÓN CORREGIDA =====

    if (!this.initialized || !this.drive) {

      throw new Error('Google Drive service no está inicializado');

    }

    // =====

    logger.info('Descargando archivo desde Drive', { fileId });

    const response = await this.drive.files.get(

      {

        fileId: fileId,

        alt: 'media', // Importante: obtener el contenido, no metadata

      },

      {

        responseType: 'arraybuffer', // Recibir como array buffer

      }

    );

    const buffer = Buffer.from(response.data as ArrayBuffer);

    logger.info('Archivo descargado exitosamente desde Drive', {

      fileId,

      size: buffer.length

    });

    return buffer;

```

```

} catch (error: any) {

  logger.error('Error al descargar archivo de Drive:', {

    fileId,

    error: error.message,

    code: error.code

  });

  // Si el token expiró, intentar refrescar

  if (error.code === 401 && this.oauth2Client) {

    logger.info('Token expirado al descargar, intentando refrescar...');

    try {

      const { credentials } = await this.oauth2Client.refreshAccessToken();

      this.oauth2Client.setCredentials(credentials);

      logger.info('Token refrescado, reintentando descarga...');

      // Reintentar descarga

      return this.downloadFileBuffer(fileId);

    } catch (refreshError) {

      logger.error('Error al refrescar token durante descarga:', refreshError);

    }

  }

  throw new Error(`Error al descargar archivo: ${error.message}`);

}

}

export default new DriveService();

.

. // =====

// src/services/queue.service.ts

// =====

import { Queue, QueueEvents } from 'bullmq';

import { config } from '../config/env';

import logger from '../utils/logger';

export interface AudioJobData {

  audioid: string;

  userId: string;

  driveFileId: string;

  driveFileUrl: string;

  filename: string;

}

class QueueService {

  public queue: Queue<AudioJobData>;

```

```

private queueEvents: QueueEvents;

constructor() {
  const connection = {
    host: config.redis.host,
    port: config.redis.port,
  };

  this.queue = new Queue<AudioJobData>(config.queue.name, {
    connection,
    defaultJobOptions: {
      attempts: config.queue.retryAttempts,
      backoff: {
        type: 'exponential',
        delay: config.queue.retryDelay,
      },
      removeOnComplete: {
        count: 100,
        age: 24 * 3600,
      },
      removeOnFail: {
        count: 50,
        age: 7 * 24 * 3600,
      },
    },
  });

  this.queueEvents = new QueueEvents(config.queue.name, { connection });
  this.setupEventListeners();
}

private setupEventListeners(): void {
  this.queueEvents.on('completed', ({ jobId }) => {
    logger.info('Job completado exitosamente', { jobId });
  });

  this.queueEvents.on('failed', ({ jobId, failedReason }) => {
    logger.error('Job falló', { jobId, failedReason });
  });
}

async addAudioJob(data: AudioJobData): Promise<string> {
  try {
    const job = await this.queue.add('process-audio', data, {
      jobId: data.audioid,
    });
  }

```

```

    logger.info('Audio agregado a la cola', {
      jobId: job.id,
      audioid: data.audioid,
    });

    return job.id!;
  } catch (error: any) {
    logger.error('Error al agregar job a la cola:', error);
    throw new Error(`Error al encolar audio: ${error.message}`);
  }
}

async getQueueMetrics(): Promise<any> {
  try {
    const [waiting, active, completed, failed, delayed] = await Promise.all([
      this.queue.getWaitingCount(),
      this.queue.getActiveCount(),
      this.queue.getCompletedCount(),
      this.queue.getFailedCount(),
      this.queue.getDelayedCount(),
    ]);

    return { waiting, active, completed, failed, delayed };
  } catch (error: any) {
    logger.error('Error al obtener métricas de la cola:', error);
    return null;
  }
}

async removeJob(jobId: string): Promise<void> {
  try {
    const job = await this.queue.getJob(jobId);

    if (job) {
      await job.remove();
      logger.info('Job removido de la cola', { jobId });
    }
  } catch (error: any) {
    logger.error('Error al remover job:', error);
  }
}

async close(): Promise<void> {
  await this.queue.close();
  await this.queueEvents.close();
}

```

```

export default new QueueService();

// =====

// src/app/models/audio.model.ts

// =====

export enum AudioStatus {

  PENDING = 'PENDING',

  PROCESSING = 'PROCESSING',

  DONE = 'DONE',

  ERROR = 'ERROR'

}

export interface Audio {

  id: string;

  filename: string;

  originalFilename: string;

  status: AudioStatus;

  transcriptionText?: string;

  driveTranscriptionFileId?: string; // Nuevo campo para el ID del archivo de
  transcripción en Google Drive

  driveTranscriptionFileUrl?: string; // Nuevo campo para la URL del archivo de
  transcripción en Google Drive

  errorMessage?: string;

  fileSize?: number;

  duration?: number;

  createdAt: string;

  updatedAt: string;

  processingStarted?: string;

  processingFinished?: string;

  user?: { // Nuevo campo para la información del usuario

    email: string;

  };

}

export interface AudioUploadResponse {

  success: boolean;

  message: string;

  data: {

    id: string;

    filename: string;

    status: AudioStatus;

    createdAt: Date;

  };

}

export interface AudioListResponse {

  success: boolean;

  data: Audio[];

```

```

pagination: {

  total: number;

  limit: number;

  offset: number;

};

}

export interface StatsResponse {

  success: boolean;

  data: {

    audios: {

      total: number;

      pending: number;

      processing: number;

      done: number;

      error: number;

    };

    queue?: {

      waiting: number;

      active: number;

      completed: number;

      failed: number;

    };

  };

}

// =====

// src/config/env.ts

// =====

import dotenv from 'dotenv';

import Joi from 'joi';

dotenv.config();

const envSchema = Joi.object({

  NODE_ENV: Joi.string().valid('development', 'production',
    'test').default('development'),

  PORT: Joi.number().default(3000),

  API_PREFIX: Joi.string().default('/api/v1'),

  DATABASE_URL: Joi.string().required(),

  REDIS_URL: Joi.string().required(),

  REDIS_HOST: Joi.string().default('localhost'),

  REDIS_PORT: Joi.number().default(6379),

  GOOGLE_DRIVE_FOLDER_ID: Joi.string().required(),

  // Actualiza estos campos de Google:

  GOOGLE_CLIENT_ID: Joi.string().required(),

  GOOGLE_CLIENT_SECRET: Joi.string().required(),

```

```

GOOGLE_REFRESH_TOKEN: Joi.string().required(),

TRANSCRIPTOR_URL: Joi.string().uri().required(),

AUTH_VERIFY_URL: Joi.string().uri().required(),

FRONTEND_BASE_URL: Joi.string().uri().required(),

CORS_ORIGINS: Joi.string().required(),

CALLBACK_SECRET: Joi.string().min(32).required(),

CALLBACK_PATH: Joi.string().default('/api/v1/callback/transcription'),

MAX_DAILY_UPLOADS: Joi.number().default(5),

MAX_FILE_SIZE_MB: Joi.number().default(50),

ALLOWED_AUDIO_TYPES:
Joi.string().default('audio/mpeg,audio/wav,audio/mp4'),

QUEUE_NAME: Joi.string().default('audioQueue'),

QUEUE_CONCURRENCY: Joi.number().default(5),

QUEUE_RETRY_ATTEMPTS: Joi.number().default(3),

QUEUE_RETRY_DELAY: Joi.number().default(5000),

JWT_SECRET: Joi.string().min(32).required(),

LOG_LEVEL: Joi.string().valid('error', 'warn', 'info', 'debug').default('info'),

LOG_FILE_PATH: Joi.string().default('/logs'),

RATE_LIMIT_WINDOW_MS: Joi.number().default(900000),

RATE_LIMIT_MAX_REQUESTS: Joi.number().default(100),
)).unknown();

const { error, value: envVars } = envSchema.validate(process.env);

if (error) {

  throw new Error(` Config validation error: ${error.message}` );

}

export const config = {

  env: envVars.NODE_ENV as string,

  port: envVars.PORT as number,

  apiPrefix: envVars.API_PREFIX as string,

  database: {

    url: envVars.DATABASE_URL as string,

  },

  redis: {

    url: envVars.REDIS_URL as string,

    host: envVars.REDIS_HOST as string,

    port: envVars.REDIS_PORT as number,

  },

  google: {

    serviceAccountPath: envVars.GOOGLE_SERVICE_ACCOUNT_JSON_PATH as
string,

    driveFolderId: envVars.GOOGLE_DRIVE_FOLDER_ID as string,

    clientId: envVars.GOOGLE_CLIENT_ID as string,

    clientSecret: envVars.GOOGLE_CLIENT_SECRET as string,

    refreshToken: envVars.GOOGLE_REFRESH_TOKEN as string,

```

```

},

services: {

  transcriptorUrl: envVars.TRANSCRIPTOR_URL as string,

  authVerifyUrl: envVars.AUTH_VERIFY_URL as string,

},

frontend: {

  baseUrl: envVars.FRONTEND_BASE_URL as string,

  corsOrigins: (envVars.CORS_ORIGINS as string).split(','),

},

callback: {

  secret: envVars.CALLBACK_SECRET as string,

  path: envVars.CALLBACK_PATH as string,

},

limits: {

  maxDailyUploads: envVars.MAX_DAILY_UPLOADS as number,

  maxFileSizeMB: envVars.MAX_FILE_SIZE_MB as number,

  allowedAudioTypes: (envVars.ALLOWED_AUDIO_TYPES as string).split(','),

},

queue: {

  name: envVars.QUEUE_NAME as string,

  concurrency: envVars.QUEUE_CONCURRENCY as number,

  retryAttempts: envVars.QUEUE_RETRY_ATTEMPTS as number,

  retryDelay: envVars.QUEUE_RETRY_DELAY as number,

},

jwt: {

  secret: envVars.JWT_SECRET as string,

},

logging: {

  level: envVars.LOG_LEVEL as string,

  filePath: envVars.LOG_FILE_PATH as string,

},

rateLimit: {

  windowMs: envVars.RATE_LIMIT_WINDOW_MS as number,

  maxRequests: envVars.RATE_LIMIT_MAX_REQUESTS as number,

},

};

```